Reduced Introduction Set Computer / Pipeline / Harvard.

1. La programmation en assembleur.

empiler valeur : C ← A - B

```
PUSH   A
PUSH   B
SUB    A - B
POP    C
```

ou

```
LOAD   A
SUB    B
STORE  C
```

autre (Registre)

```
LDS   R16, A
LDS   R17, B
ADD   R16, R17
STS   C, R16
```

Mnemonics	Operands	Description	Operation	Flags	#Clocks
Arithmetic and Logic Instructions					
ADD	Rd, Rr	Add two registers	Rd ← Rd + Rr	Z,C,N,VH	1
ADC	Rd, Rr	Add with carry two registers	Rd ← Rd + Rr + C	Z,C,N,V,H	1
ADIW	Rdl, K	Add immediate to word	Rdh: Rdl ← Rdh: Rdl + K	Z,C,N,V,S	2
SUB	Rd, Rr	Subtract two registers	Rd ← Rd – Rr	Z,C,N,V,H	1
SUBI	Rd, K	Subtract constant from register	Rd ← Rd – K	Z,C,N,V,H	1
SBC	Rd, Rr	Subtract with carry two registers	Rd ← Rd – Rr – C	Z,C,N,V,H	1
SBCI	Rd, K	Subtract with carry constant from reg.	Rd ← Rd – K – C	Z,C,N,V,H	1
SBIW	Rdl, K	Subtract immediate from word	Rdh: Rdl ← Rdh: Rdl – K	Z,C,N,V,S	2
AND	Rd, Rr	Logical AND registers	Rd ← Rd × Rr	Z,N,V	1
ANDI	Rd, K	Logical AND register and constant	Rd ← Rd × K	Z,N,V	1
OR	Rd, Rr	Logical OR registers	Rd ← Rd v Rr	Z,N,V	1
ORI	Rd, K	Logical OR register and constant	Rd ← Rd v K	Z,N,V	1
EOR	Rd, Rr	Exclusive OR registers	Rd ← Rd ⊕ Rr	Z,N,V	1
COM	Rd	One's complement	Rd ← 0xFF – Rd	Z,C,N,V	1
NEG	Rd	Two's complement	Rd ← 0x00 – Rd	Z,C,N,V,H	1

Jeu d'instruction (1)

- ⇨ Instruction d'affectation : MOV
  (Transfert CPU Mémoire)
- ⇨ Instructions arithmétiques
  (Opération Acc / Donnée)
  : INC (incrémentation)
  : DEC (décrementation)
  : ADD (addition)
  : SUB (soustraction)
  : CMP (soustraction sans sauvegarde)
  : NEG
- ⇨ Instructions logiques
  : NOT, OR, XOR
  : AND, TEST (= AND sans sauvegarde)
  : SHL (SHR), SAL (SAR)
  : ROL (ROR), RCL (RCR)

Jeu d'instruction (2)

- ⇨ Branchement : JMP
- ⇨ Branchements conditionnels : JE/ JZ (JNE/ JNZ) : Jump if zero
  : JO (JNO) : Jump if overflow
  : JS (JNS) : Jump if sign
- ⇨ Comparaison de valeurs

CMP AX, BX suivi d 'un test :

	(entiers naturels)	(complément à 2)
AX > BX ?	JA (>= JAE)	JG (JGE)
AX < BX ?	JB (JBE)	JL (JLE)

adressage immédiat :

```
68hc711   : LDAA  #5
MAM       : LOAD  #5
Atmega2560: LDI   R17, 20
```

R17 ← 20

adressage direct :

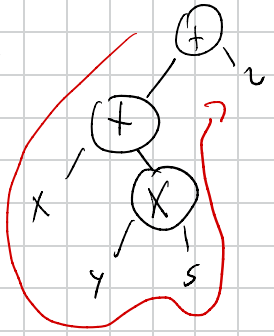
```
68hc711   : LDAA  $
MAM       : LOAD  $1000
Atmega2560: LDS   R16, 0x1000
```

indicelk :

```
68hc711   : LDAA  ,X
MAM       : LOAD  0,X
Atmega2560: ST    Y, AS
```

(X + (Y x S)) + 2

```
PUSH   X, Y;
PUSHI  S
MUL
PLUS
PUSHI  2
PLUS
```



$E = ((y+5)+x)+2$.

```
LOAD   1      direct
MULI   5      immediat
ADD    0      direct
ADDI   2      immediat
STORE  2      address
```

$E = (x+1)-(5+y)$

```
LOAD   0          LOAD 3
ADDI   1          SUB  4
STORE  3          STORE 2
LOADI  5
MUL    1
STORE  4
```

ou $x, y, E \to 0, 1, 2$.

ou

```
LOADI  5
MUL    1
STORE  2
LOAD   0
ADDI   1
SUB    2
STORE  2
```

$E = (x + (z - 2) \times 4) + y \times 3$.

0	x
1	y
2	z
3	E

```
LOADI  3
MUL    1
STORE  4

LOADI  2
SUBI   2
MUL    4
ADD    0

ADD    4
```

Université Côte d'Azur
Licence Électronique
Enseignement : SLEE600 - Microprocesseurs

TD1

Evaluation d'expressions arithmétiques selon l'architecture sous-jacente

Préambule

L'objectif de ce TD, est de vous convaincre que l'implémentation d'un algorithme donné est fortement dépendant du type d'architecture qui effectue les manipulations de données.

1 Machine à pile

Considérons l'expression : $\Delta = b^2 - 4ac$, codée sur des entiers signés 16 bits en complément à 2.

1. Dessinez l'arbre d'évaluation. Dans cet arbre, chaque noeud représente un opérateur arithmétique et chaque branche symbolise une des opérandes. Les opérateurs admis sont "*" (noté MULT),"+" (ADD),"-" (SUB).

2. Notre machine à pile dispose des trois opérateurs précédents MULT, ADD, SUB et deux instructions de manipulation de la pile (PUSH et POP). Notons que chaque opérateur arithmétique *op* est en fait évalué comme :

(POP *op* POP) PUSH

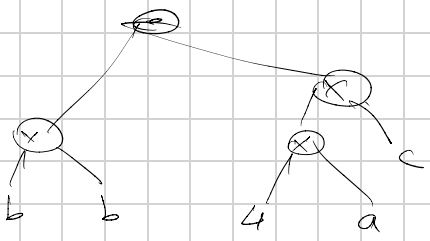
3. Écrivez un programme qui évalue Δ.

4. Dessinez le contenu de la pile lors de l'exécution.

2 Machine à accumulateur

1. À partir de l'arbre d'évaluation, déterminez le nombre de cases mémoires nécessaire pour évaluer l'expression précédente.
   — sans rien optimiser,

$\Delta = b^2 - 4ac$



PILE

Push b	9
Push a	8
MUL	
Push 4	4
Push a	a
MUL	
Push c	c
MUL	
Sub	

ACCUMULATEUR

a	0
b	1
c	2

LOADI	4
MUL	0
MUL	2
Store	3
LOAD	1
MUL	1
Sub	3
Store	3

le registre état du processeur. (SREG)

machine à pile

machine à accumulateur

SREG [V C N Z]

V ↑ overflow

C ↑ retenue

N ↑ négatif [complément à 1 + 1]

Z ↑ zéro

Load #0 => registre immédiat.

— en minimisant la taille de la mémoire et le nombre de transfert mémoire - accumulateur,

2. Ecrivez dans les deux cas, le pseudo-code correspondant. Chaque opérateur arithmétique *op* prend un paramètre x et effectue l'opération :

```
Regs[Acc] <-- Regs[Acc]  op  x
```

x peut être une constante ou une donnée rangée en mémoire (ex : Mem[100]).

Jeux d'instructions de la machine à Accumulateur

Transfert de données (nb. cycles/taille instruction)

inst.	opération	opcode	Inh	Imm	Dir	InDir
LOAD	$Reg[Acc] \leftarrow Mem[Adr]$	00000XX			9/3	9/3
LOADI	$Reg[Acc] \leftarrow const$	0000001		5/2		
STORE	$Mem[Adr] \leftarrow Reg[Acc]$	00001XX			8/3	8/3
TAXL	$Reg[Xlow] \leftarrow Reg[Acc]$	00010XX	4/1			
TAXH	$Reg[Xhigh] \leftarrow Reg[Acc]$	00011XX	4/1			
TXLA	$Reg[Acc] \leftarrow Reg[Xlow]$	00100XX	4/1			
TXHA	$Reg[Acc] \leftarrow Reg[Xhigh]$	00101XX	4/1			
XAXL	$tmp \leftarrow Reg[Acc]$ $Reg[Acc] \leftarrow Reg[Xlow]$ $Reg[Xlow] \leftarrow tmp$	00110XX	5/1			
XAXH	$tmp \leftarrow Reg[Acc]$ $Reg[Acc] \leftarrow Reg[Xhigh]$ $Reg[Xhigh] \leftarrow tmp$	00111XX	5/1			

Opérations arithmétiques et logiques (nb. cycles/taille instruction)

inst.	opération	opcode	Inh	Imm	Dir	InDir
MUL	$Reg[Acc] \leftarrow Reg[Acc] * Mem[Adr]$	01000XX			9/3	9/3
MULI	$Reg[Acc] \leftarrow Reg[Acc] * const$	0100001		5/2		
ADD	$Reg[Acc] \leftarrow Reg[Acc] + Mem[Adr]$	01001XX			9/3	9/3
ADDI	$Reg[Acc] \leftarrow Reg[Acc] + const$	0100101		5/2		
SUB	$Reg[Acc] \leftarrow Reg[Acc] - Mem[Adr]$	01010XX			9/3	9/3
SUBI	$Reg[Acc] \leftarrow Reg[Acc] - const$	0101001		5/2		
CMP	$\leftarrow Reg[Acc] - Mem[Adr]$	01011XX			9/3	9/3
CMPI	$\leftarrow Reg[Acc] - const$	0101101		5/2		
NEG	$Reg[Acc] \leftarrow -Reg[Acc]$	01100XX	4/1			
AND	$Reg[Acc] \leftarrow Reg[Acc] \& Mem[Adr]$	01101XX			9/3	9/3
ANDI	$Reg[Acc] \leftarrow Reg[Acc] \& const$	0110101		5/2		
OR	$Reg[Acc] \leftarrow Reg[Acc] \mid Mem[Adr]$	01110XX			9/3	9/3
ORI	$Reg[Acc] \leftarrow Reg[Acc] \mid const$	0111001		5/2		
XOR	$Reg[Acc] \leftarrow Reg[Acc] \wedge Mem[Adr]$	01111XX			9/3	9/3
XORI	$Reg[Acc] \leftarrow Reg[Acc] \wedge const$	0111101		5/2		
NOT	$Reg[Acc] \leftarrow \sim Reg[Acc]$	10000XX	4/1			
ASR	$C \leftarrow Reg[Acc][0]$ $Reg[Acc] \leftarrow Reg[Acc]/2$	10001XX	4/1			
LSR	$C \leftarrow Reg[Acc][0]$ $Reg[Acc] \leftarrow Reg[Acc] >> 1$	10010XX	4/1			
LSL	$C \leftarrow Reg[Acc][7]$ $Reg[Acc] \leftarrow Reg[Acc] << 1$	10011XX	4/1			

Branchements (nb. cycles :vrai,faux/taille instruction)

inst.	opération	opcode	Inh	Imm	Dir	InDir
BEQ	*si Z alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	10100XX		7,3/3		
BNE	*si Z alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	10101XX		7,3/3		
BLT	*si N alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	10110XX		7,3/3		
BLE	*si* $N + Z$ *alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	10111XX		7,3/3		
BGE	*si N alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11000XX		7,3/3		
BGT	*si* $N + Z$ *alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11001XX		7,3/3		
BC	*si C alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	11010XX		7,3/3		
BNC	*si C alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11011XX		7,3/3		
BV	*si V alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	11100XX		7,3/3		
BNV	*si V alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11101XX		7,3/3		
B	$Reg[PC] \leftarrow Reg[PC] + depl$	11110XX		7/3		
NOP	*aucune operation*	11111XX	3/1			